

WINDOWS FORENSICS PROJECT: ANALYZER

Project Overview

This report explains how the Bash script TCB-2506.s4.nx212.sh works. The script automates HDD and memory analysis by checking the execution user, accepting an input file, creating a results directory, installing/checking required forensic tools, extracting data with different carvers, searching for network traffic, extracting human-readable strings, running Volatility memory analysis, saving results into a report, and compressing the final evidence folder into a ZIP archive.

The explanation below follows the exact project requirements: automated HDD and memory analysis, Volatility memory analysis, final results, and creativity improvements.

1. Automate HDD and Memory Analysis

1.1. Check the current user; exit if not root.

Code:

```
if [ "$(whoami)" != "root" ]
then
    echo -e "${RED}[!] Must run as root. Exiting the script.${NC}"
    exit
fi
```

The script checks the current user with whoami. If the user is not root, the script prints an error message and exits. This is important because forensic tools such as carving tools, package installation, and memory analysis may require administrator privileges.

1.2. Allow the user to specify the filename; check if the file exists.

Code:

```
read -p "[*] Enter the filename you want to analyze: " FILE

if [ -f "$FILE" ]
then
    echo -e "${GREEN}[+] File exists: $FILE${NC}"
else
    echo -e "${RED}[!] File does not exist. Exiting.${NC}"
    exit
fi
```

The script asks the user to enter the name of the file that will be analyzed. The value is saved in the FILE variable. Then the script checks whether the file exists using -f. If the file does not exist, the script stops to avoid running forensic tools on an invalid input.

1.3. Create a function to install the forensic tools if missing.

Code:

```
apt-get update > "$OUTPUT_DIR/install_log.txt" 2>&1
apt-get install -y binutils foremost binwalk bulk-extractor zip >> "$OUTPUT_DIR/install_log.txt"
2>&1
```

The INSTALL_TOOLS function updates the package information and installs/checks the required tools. The package binutils provides the strings command, foremost and binwalk are used for carving and extraction, bulk-extractor is used for extracting forensic artifacts, and zip is used to archive the final results. The installation output is saved into install_log.txt so the terminal remains clean.

1.4. Use different carvers to automatically extract data.

Code:

```
foremost -i "$FILE" -o "$OUTPUT_DIR/carving/foremost" > /dev/null 2>&1
binwalk -e -C "$OUTPUT_DIR/carving/binwalk" --run-as=root "$FILE" > /dev/null 2>&1
bulk_extractor -o "$OUTPUT_DIR/carving/bulk_extractor" "$FILE" > /dev/null 2>&1
```

The RUN_CARVERS function uses three different tools to extract data automatically. Foremost extracts files based on file headers and footers. Binwalk searches for and extracts embedded files. Bulk Extractor extracts forensic artifacts from the input file. The output of these tools is hidden from the terminal to keep the script output readable.

1.5. Data should be saved into a directory.

Code:

```
OUTPUT_DIR="Analysis_Results_$(date +%Y%m%d_%H%M%S)"
mkdir "$OUTPUT_DIR"

REPORT="$OUTPUT_DIR/report.txt"
mkdir "$OUTPUT_DIR/carving"
mkdir "$OUTPUT_DIR/strings"
mkdir "$OUTPUT_DIR/volatility"
```

The script creates a unique output directory using the current date and time. This prevents overwriting old results. Inside this directory, the script creates separate folders for carving results, strings results, Volatility results, logs, the SHA-256 hash file, and the report file.

1.6. Attempt to extract network traffic; if found, display location and size.

Code:

```
location=$(find "$OUTPUT_DIR" -type f -name "packets.pcap" 2>/dev/null | head -n 1)
```

```

if [ -f "$location" ]
then
    pcap_name=$(basename "$location")
    size=$(du -h "$location" | cut -f1)

    echo -e "${GREEN}[+] Network traffic file found.${NC}"
    echo -e "${YELLOW}Name: $pcap_name${NC}"
    echo -e "${YELLOW}Location: $location${NC}"
    echo -e "${YELLOW}Size: $size${NC}"
fi

```

The FIND_NETWORK_TRAFFIC function searches inside the output directory for packets.pcap, which is commonly created by Bulk Extractor when network traffic is extracted. If the file is found, the script displays the file name, location, and size to the user and also saves this information in the report.

1.7. Check for human-readable strings such as exe files, passwords and usernames.

Code:

```

strings "$FILE" | grep -i "exe" > "$OUTPUT_DIR/strings/Strings_exe.txt"
strings "$FILE" | grep -i "password" > "$OUTPUT_DIR/strings/Strings_password.txt"
strings "$FILE" | grep -i "username" > "$OUTPUT_DIR/strings/Strings_username.txt"
strings "$FILE" | grep -i "user" > "$OUTPUT_DIR/strings/Strings_user.txt"
strings "$FILE" | grep -i "http" > "$OUTPUT_DIR/strings/Strings_http.txt"
strings "$FILE" | grep -E "[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}" >
"$OUTPUT_DIR/strings/Strings_email.txt"

```

The RUN_STRINGS function extracts human-readable text from the input file and searches for interesting values. The script creates separate output files for EXE-related strings, passwords, usernames, users, HTTP/URL strings, and email addresses. The email search uses a regular expression to find realistic email patterns instead of only searching for the @ symbol.

2. Memory Analysis with Volatility

2.1. Check if the file can be analyzed in Volatility; if yes, run Volatility.

Code:

```

VOL="./vol"

if [ ! -f "$VOL" ]
then
    echo -e "${RED}[!] Volatility file ./vol not found.${NC}"
    echo "Volatility file ./vol not found." >> "$REPORT"
    return
fi

"$VOL" -f "$FILE" imageinfo > "$OUTPUT_DIR/volatility/imageinfo.txt" 2>&1

```

The RUN_VOLATILITY function first checks whether the local Volatility executable named vol exists. If it is missing, the function stops but the whole script continues. If Volatility is available, the script runs imageinfo to check whether the file can be analyzed as a memory image.

2.2. Find the memory profile and save it into a variable.

Code:

```
PROFILE=$(grep "Suggested Profile" "$OUTPUT_DIR/volatility/imageinfo.txt" | cut -d ":" -f2 | cut -d "," -f1 | xargs)

if [ "$PROFILE" = "" ]
then
    echo -e "${RED}[!] Profile not found.${NC}"
    echo "Profile not found." >> "$REPORT"
    return
fi
```

The script reads the imageinfo output and extracts the first suggested Volatility profile. The profile is saved into the PROFILE variable. If no profile is found, the Volatility analysis is skipped because commands such as pslist, connscan, and hivelist need a valid profile.

2.3. Display the running processes.

Code:

```
"$VOL" -f "$FILE" --profile="$PROFILE" pslist > "$OUTPUT_DIR/volatility/pslist.txt" 2>&1
```

The script runs the pslist plugin to extract the running processes from the memory dump. The result is saved in pslist.txt inside the Volatility results directory. This allows the investigator to review the active processes found in memory.

2.4. Display network connections.

Code:

```
"$VOL" -f "$FILE" --profile="$PROFILE" connscan > "$OUTPUT_DIR/volatility/connscan.txt" 2>&1
```

The script runs the connscan plugin to extract network connection information from the memory image. The output is saved in connscan.txt. This helps the investigator identify possible active or historical network connections found in memory.

2.5. Attempt to extract registry information.

Code:

```
"$VOL" -f "$FILE" --profile="$PROFILE" hivelist > "$OUTPUT_DIR/volatility/hivelist.txt" 2>&1
```

The script runs the hivelist plugin to attempt to extract registry hive information from the memory dump. The output is saved in hivelist.txt. This meets the requirement to attempt registry extraction during memory analysis.

3. Results

3.1. Display general statistics.

Code:

```
END_TIME=$(date)
FOUND_FILES=$(find "$OUTPUT_DIR" -type f | wc -l)

echo -e "${YELLOW}Start time: $START_TIME${NC}"
echo -e "${YELLOW}End time: $END_TIME${NC}"
echo -e "${YELLOW}Number of result files: $FOUND_FILES${NC}"
```

The FINAL_STATISTICS function records the end time of the analysis and counts all files created inside the output directory. The script displays the start time, end time, and number of result files to the user.

3.2. Save all the results into a report.

Code:

```
echo "Simple Forensic Analyzer Report" > "$REPORT"
echo "Analyzed file: $FILE" >> "$REPORT"
echo "Start time: $START_TIME" >> "$REPORT"
echo "File carving results:" >> "$REPORT"
echo "Strings Analysis:" >> "$REPORT"
echo "Volatility Analysis:" >> "$REPORT"
echo "Final Statistics" >> "$REPORT"
```

The script creates report.txt in the output directory and appends important information during each analysis step. The report contains the analyzed filename, start time, tool results, extracted file locations, network traffic details, Volatility result paths, final statistics, and ZIP archive name.

3.3. Zip the extracted files and the report file.

Code:

```
echo "ZIP archive: $OUTPUT_DIR.zip" >> "$REPORT"
zip -r "$OUTPUT_DIR.zip" "$OUTPUT_DIR" > /dev/null 2>&1
```

The ZIP_RESULTS function compresses the whole output directory into a ZIP archive. This archive includes the report file, hash file, strings results, carved files, Volatility results, and logs. The zip output is hidden to keep the terminal clean.

4. Creativity

I added several creativity improvements to make the project more useful and easier to present.

First, the script calculates a SHA-256 hash of the analyzed file. This is useful in digital forensics because it helps verify the integrity of the evidence. If the file changes, the hash will change as well.

Code:

```
sha256sum "$FILE" > "$OUTPUT_DIR/file_hash.txt"
```

Second, the email search uses a regular expression to find realistic email patterns instead of only searching for the @ symbol. This improves the quality of the strings analysis because it focuses on email-like values instead of saving every line that simply contains @.

Third, I added colored terminal output. Cyan is used for running steps, green for successful results, red for errors, yellow for important findings, and magenta for section headers. This makes the script easier for the investigator to follow during execution.

Fourth, I added an ASCII banner at the start of the script to make the tool look more professional during a demonstration. The banner idea was inspired by Rotem Bacal / R0t3m's GhostLogin project post on LinkedIn, and the credit link is included as a comment in the script.

Finally, I added credit comments for the Binwalk command structure because it was inspired by Rotem Bacal / R0t3m's public Analyzer.sh project.

Testing the Script

The script can be tested in Kali Linux from the project directory. The local Volatility executable named vol should be in the same folder as the script if memory analysis is required.

Code:

```
chmod +x TCB-2506.s4.nx212.sh
sudo ./TCB-2506.s4.nx212.sh
# Example input file:
memdump.mem
```

After the analysis finishes, the user should see a results directory and a ZIP archive. The report.txt file inside the results directory contains the main findings and paths to extracted artifacts.

Proof of Functionality

The proof of functionality is provided through the script output and the created result files. The terminal output shows the file validation, SHA-256 hash calculation, tool checking, file carving, network traffic detection, strings analysis, Volatility profile detection, process extraction, network connection extraction, registry extraction, final statistics, and ZIP archive creation.

```
(kali@kali)-[~/Desktop/WindowsForensics_FinalProject]
$ sudo bash TCB-2506.s4.nx212.sh
[sudo] password for kali:
```



Forensics Analyzer by B3awrot

```
[*] Enter the filename you want to analyze: memdump.mem
[+] File exists: memdump.mem
[+] Output directory created: Analysis_Results_20260603_043442
[*] Calculating SHA-256 file hash...
[+] SHA-256 file hash saved in: Analysis_Results_20260603_043442/file_hash.txt
[*] Installing/checking required tools...
[+] Tools are installed/checked.
```

FILE CARVING

```
[*] Running carving tools...
[*] 1. Running foremost...
[*] 2. Running binwalk...
[*] 3. Running bulk_extractor...
[+] File carving finished.
```

NETWORK TRAFFIC

```
[*] Searching for network traffic file...
[+] Network traffic file found.
```

Name: packets.pcap

Location: Analysis_Results_20260603_043442/carving/bulk_extractor/packets.pcap

Size: 104K

STRINGS ANALYSIS

```
[*] Running strings analysis...
[+] Strings analysis finished.
```

VOLATILITY ANALYSIS

```
[*] Running Volatility analysis...
[*] Checking if the file can be analyzed with Volatility...
[+] Profile found: WinXPSP2x86
[*] Extracting running processes...
[*] Extracting network connections...
[*] Extracting registry information...
[+] Volatility analysis finished.
```

FINAL STATISTICS

```
[*] Creating final statistics...
```

Start time: Wed Jun 3 04:34:37 AM EDT 2026

End time: Wed Jun 3 04:36:59 AM EDT 2026

Number of result files: 3381

ZIP RESULTS

```
[*] Creating ZIP archive...
[+] ZIP archive created: Analysis_Results_20260603_043442.zip
```

ANALYSIS COMPLETED

```
[+] Analysis finished successfully.
```

Results directory: Analysis_Results_20260603_043442

Report file: Analysis_Results_20260603_043442/report.txt

ZIP archive: Analysis_Results_20260603_043442.zip

kali

Desktop

WindowsForensics_FinalProject



Analysis_Results_20260603_043442



memdump.mem



dump2.mem



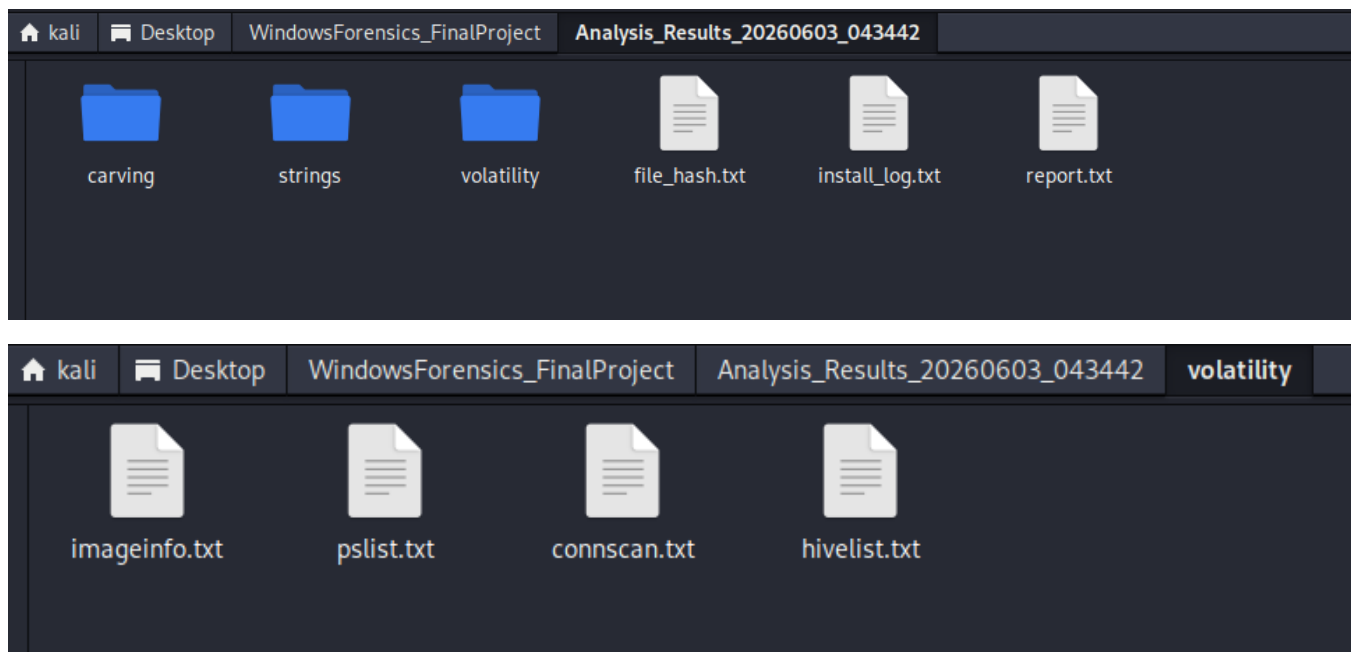
vol



TCB-2506.s4.nx212.sh



Analysis_Results_20260603_043442.zip



Conclusion

The TCB-2506.s4.nx212.sh script meets the project requirements by automating HDD and memory analysis, using different forensic tools, saving all results in an organized directory, extracting network traffic information when available, searching for human-readable sensitive information, running Volatility plugins, generating a report, and creating a final ZIP archive. The project also includes creativity features such as SHA-256 hashing, colored output, an ASCII banner, and proper credit for external inspiration.

